RAG Service

Given the comprehensive nature of the provided codebase, generating “ultra-detailed” documentation for *every* file in a single response would be excessively long and difficult to navigate.

Instead, I have structured this documentation guide into two main parts to perfectly fulfill your request:

1. **The Master Index (“What is What, What it Does, and Where”):** A conceptual mapping of the system’s features to the specific files and code blocks that execute them.
2. **Detailed Component Documentation:** Ultra-detailed breakdowns of the core logic files, configuration, API layer, and the testing framework.

Part 1: The Master Index (Functional Mapping)

This section explains high-level system concepts and maps them directly to the code implementing them.

Concept / Process	What it Does	Relevant Files	Key Code Blocks / Functions
System Source of Truth	Defines the mandatory application settings and the strict schema governing all documents.	`config.py`	`Settings` class, `METADATA_SCHEMA` list.
Dynamic Schema Enforce.	Ensures incoming API data matches the allowed metadata fields before processing.	`models.py`, `config.py`	`models.py`: `_validate_metadata()` helper, `ChunkIn` and `SearchRequest` validators.
Index Mapping Gen.	Programmatically generates the complex Elasticsearch index mapping JSON based on the dynamic schema.	`index_manager.py`	`build_index_mapping()`, `get_dynamic_mapping()`.
Request Context (Tracing)	Captures or generates a unique ID (Correlation ID) for every incoming HTTP request and attaches it to logs.	`middleware.py`, `logging_config.py`	`middleware.py`: `correlation_and_access_log()`. `logging_config.py`: `CORRELATION_ID_CTX`, `JsonFormatter`.
Text Normalization	Normalizes raw text (whitespace removal, case normalization) to ensure deterministic hash generation.	`hashing.py`	`normalize_text()`, `content_hash()`.
Semantic Routing (Dedup)	Implementation of the logic to decide whether a chunk is new, modified (metadata only), or a duplicate to save compute.	`ingestion.py`	`ingest_chunks()` (Pass 1 and Pass 2), `_existing_docs_for_hash()`, `_build_upsert_actions()`.
Bulk Data Ingestion	Efficiently indexes or updates many documents at once using Elasticsearch's bulk API.	`ingestion.py`, `es_client.py`	`ingestion.py`: `_apply_bulk_operations()`. `es_client.py`: `bulk()`.
Embedding Generation	Handles the asynchronous HTTP communication with the external AI router service to generate vectors.	`embeddings.py`	`RouterEmbedder.embed()`, `RouterEmbedder._call_router()`.

Concept / Process	What it Does	Relevant Files	Key Code Blocks / Functions
Hybrid Search	Combines vector similarity (KNN) and keyword search (BM25) while strictly applying metadata filters.	`search.py`	`hybrid_search()`, `build_hybrid_query()`, `apply_filters_to_query()`.
Unified Error Handling	Catches all domain-specific exceptions, logs them with context, and returns standardized JSON responses.	`exceptions.py`, `middleware.py`	`exceptions.py`: `RagError` hierarchy. `middleware.py`: `rag_exception_handler()`.
Async ES Management	Manages the lifecycle and connection pool of the `AsyncElasticsearch` client.	`es_client.py`	`get_es_client()`, `on_startup`, `on_shutdown` events in `main.py`.
Test Mocks	Provides fake implementations of external services (ES and Embedder) for fast, offline testing.	`conftest.py`	`FakeAsyncES`, `FakeEmbedder` fixtures.

Part 2: Detailed Component Documentation

A. Configuration & Schema (The Blueprint)

File: `config.py`

This file is the single source of truth for the entire application. Modifying settings here alters the behavior of ingestion validation, Elasticsearch mapping creation, and test behavior.

- **Imports:**
  - `pydantic_settings`: For loading environment variables into typed models.
  - `logging`: To configure the application root logger.
- **Key Data Structures:**
  - `MetadataField` **(Class)**: A helper class defining the structure of an allowed metadata field.
    - `name`: The key name as it appears in API payloads and inside ES `metadata.*`.
    - `es_type`: The Elasticsearch data type (e.g., `keyword`, `text`, `integer`).
    - `pydantic_type`: The expected Python type for validation (e.g., `str`, `int`).
  - `METADATA_SCHEMA` **(List)**: The central definition of all valid metadata filters.
    - *Details*: Any field added here automatically becomes:
      1. A sub-field of `metadata.*` in Elasticsearch mappings.
      2. A valid key in the `metadata` dictionary of an ingestion request.
      3. A valid key in the `filters` dictionary of a search request.
- **Key Class:** `Settings`
  - Inherits from `BaseSettings` (pydantic). It loads values from `.env` or system environment variables.
  - **Validation**: It validates that critical settings (like `AI_ROUTER_API_KEY`) are present during startup.
  - `ENV_MAP`: A helper dictionary translating Python names (e.g., `LOG_LEVEL`) to environment variable names (e.g., `LOGGING_LEVEL`).

B. Core Logic (The Brain)

File: `ingestion.py`

This is the most complex module in the service, responsible for orchestrating semantic routing and data persistence.

- **Logic Flow (`ingest_chunks` function):**
  1. **Validation**: Pydantic validates the `IngestRequest`.
  2. **Normalize & Hash**: Iterates through incoming chunks, calls `hashing.py` to generate hashes.
  3. **Pass 1: Semantic Routing (State Check)**:
     - Calls `_existing_docs_for_hash` to query Elasticsearch via a `terms` query on all generated hashes. This retrieves all *currently known* chunks matching the input.
     - It then iterates through the input chunks and classifies them into one of four states:
       1. `ingested_completely_new`: Hash not found in ES. Needs embedding and indexing.
       2. `ingested_new_metadata`: Hash exists, but `force_on_metadata_change=True` AND metadata differ. Needs embedding and overwriting.
       3. `skipped`: Hash exists, metadata is different, but `force_on_metadata_change=False`.
       4. `skipped_identical`: Hash and metadata are identical.
  4. **Pass 2: Embedding**:
     - A combined list is made of `ingested_completely_new` and `ingested_new_metadata`.
     - `RouterEmbedder.embed` is called *only* for this subset, saving significant compute costs.
  5. **Pass 3: Persistence**:
     - Calls `_build_upsert_actions` to create Elasticsearch bulk API directives (e.g., `{"index": {...}, "pipeline": "...", "_id": "..."}`) for the newly embedded chunks.
     - Calls `_apply_bulk_operations`, which sends the payload to `es_client.bulk()`.
  6. **Response**: Constructs and returns an `IngestionResponse` summarizing the operation.
- **Exception Handling**: It catches specific errors (`ValidationError` during Pydantic checks) and raises domain-specific exceptions (`MetadataValidationError`).

File: `search.py`

Responsible for translating abstract RAG search requests into executable Elasticsearch queries.

- **Logic Flow (`hybrid_search` function):**
  1. **Embedding**: Asynchronously calls `embedder.embed()` to convert the user's `query_text` into a dense vector.
  2. **Query Building**: Calls `build_hybrid_query()` with the text, the vector, and the metadata filters.
  3. **Filter Application (`apply_filters_to_query`)**: This helper converts the dictionary of filters (e.g., `{'doc_type': 'invoice'}`) into Elasticsearch term/terms queries.
     - *Detail*: These filters are applied **twice**: once in the `knn` section (to restrict the vector search space) and once in the `bool` / `filter` context (restricting the BM25 space).
  4. **Boosts**: Applies the `BM25_BOOST` and `KNN_BOOST` defined in `config.py` to weight the results.
  5. **Execution**: Asynchronously executes the search on the Elasticsearch cluster.
  6. **Mapping**: Iterates through the raw Elasticsearch response (`resp['hits']['hits']`), extracts the relevant data (`_score`, `_source['content']`, `_source['metadata']`), and maps them into `SearchHit` Pydantic models.
- **Exceptions**: Catches Elasticsearch client errors and translates them into `SearchError`.

C. Data Definition (The Skeleton)

File: `models.py`

Defines the Pydantic models that govern the API inputs and outputs and enforce the dynamic schema.

- **Dynamic Validation Helper: `_validate_metadata(cls, v)`**
  - This is the core validation logic. It iterates through the keys of the provided metadata dictionary.
  - It references `METADATA_SCHEMA` in `config.py`.
  - If a key is found, it ensures the *type* of the provided value matches the `pydantic_type` defined in the schema.
  - If a key is *not* found in the schema, it raises a `ValueError`.
- **Key Models:**
  - **`ChunkIn`**: Model for a single chunk within an ingestion request. It applies `_validate_metadata` to its `metadata` field.
  - **`IngestRequest`**: Model for the entire ingestion endpoint payload (list of `ChunkIn`).
  - **`SearchRequest`**: Model for the hybrid search endpoint. Its `filters` field is also validated against `METADATA_SCHEMA` to ensure users cannot filter by arbitrary fields.
  - **`SearchResponse`**: The output model, structured for clean consumption by frontend applications.

D. Testing Framework

File: `conftest.py`

This is the root configuration for the `pytest` suite. It provides critical fixtures that make offline testing possible.

- `fake_settings` **(Fixture)**: Mocks the `config.settings` object, providing deterministic values for tests, overriding environment variables.
- `FakeAsyncES` **(Class & Fixture)**:
  - This is a sophisticated mock that mimics the behavior of the `AsyncElasticsearch` client.
  - **State**: It maintains an internal `index` dictionary and `bulk_history` list.
  - `bulk(self, operations)`: Implementation that parses incoming bulk JSON directives.
    - `index` *operation*: Simulates adding or overwriting data in the internal state, recording the time.
    - *Errors*: It can be forced to return failure responses by setting the `error_hashes` property, allowing `test_ingestion_errors.py` to validate retry logic.
  - `search(self, ...)`: Implementation of the `term` query logic to simulate looking up chunks by hash.
- `FakeEmbedder` **(Class & Fixture)**:
  - Mocks the external AI Router.
  - It generates deterministic vectors based on the input text (e.g., using a seeded randomizer or text-length heuristics) rather than calling a real API. This makes tests repeatable and fast.

Architecture in 30 seconds

```
                  ┌──────────────────────────────────────┐
HTTP request ──▶ │ FastAPI router (thin)                 │
                  │   - validates body via Pydantic      │
                  │   - delegates to service             │
                  └──────────────────────────────────────┘
                                  │
                                  ▼
                  ┌──────────────────────────────────────┐
                  │ Service layer                        │
                  │   - app/ingestion.py  (dedup + write) │
                  │   - app/search.py     (hybrid query)  │
                  │ Raises RagError subclasses on failure. │
                  └──────────────────────────────────────┘
                                  │
                                  ▼
                  ┌──────────────────┐  ┌──────────────────┐
                  │ Elasticsearch    │  │ Embedder         │
                  │ (AsyncES client) │  │ (httpx → router) │
                  └──────────────────┘  └──────────────────┘
```

Cross-cutting (`app/middleware.py`):

- Correlation ID (`X-Correlation-ID`) read from header or minted; propagated via `contextvars` so logs in async tasks still see it.
- Global exception handler maps `RagError` subclasses to JSON responses with the right HTTP status and includes the correlation ID in the body.
- Per-request access log line in JSON.

Single source of truth: metadata

Edit `METADATA_SCHEMA` in `app/config.py`. That's it. Index mapping, ingestion validation, search filter validation, and the `/ingestion/schema` introspection endpoint all derive from this list.

```
METADATA_SCHEMA = [
    MetadataField("doc_id",       "keyword", required=True),
    MetadataField("doc_title",    "text",    required=True),
    MetadataField("regulation_name", "keyword"),
    MetadataField("article_number",  "keyword"),
    MetadataField("page_number",     "integer"),
    MetadataField("effective_date",  "date"),
    MetadataField("jurisdiction",    "keyword"),
    # ... add or remove fields here, nowhere else.
]
```

Tests `test_adding_a_field_propagates_*` and `test_removing_a_field_*` in `tests/test_dynamic_metadata.py` lock this contract in place.

Semantic routing (dedup) algorithm

Per chunk, on ingest:

1. `content_hash = SHA256(normalize_whitespace(text))`
2. Search the index for documents with the same `content_hash`.
3. If none → ingest as new.
4. If matches found:
   - **Default behavior** (`force_on_metadata_change=False`): skip.
   - **Force flag on**: if any existing match has equal metadata, skip; otherwise ingest as a new document with status `ingested_new_metadata`.

Why hash whitespace-normalized text? PDF re-extraction is noisy — runs of spaces and stray newlines shouldn't trigger re-ingest. Case is preserved because compliance docs are case-sensitive ("MUST" ≠ "must").

Running locally

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # then edit ES_PASSWORD etc.

# Start a local Elasticsearch (8.x) however you normally do.
# Then:
uvicorn main:app --reload --port 8000

# Sanity check:
curl localhost:8000/health
curl -X POST localhost:8000/ingestion/index
curl localhost:8000/ingestion/schema | jq

curl -X POST localhost:8000/ingestion \
  -H 'Content-Type: application/json' \
  -H 'X-Correlation-ID: my-test-001' \
  -d '{
    "chunks": [
      {"content": "Article 143(1) of the CRR ...",
       "metadata": {"doc_id": "crr-1", "doc_title": "CRR", "page_number": 1}}
    ]
  }'

curl -X POST localhost:8000/search \
  -H 'Content-Type: application/json' \
  -d '{"query": "internal ratings based approach", "top_k": 5,
       "filters": {"jurisdiction": "UK"}}'
```

Running the tests

```
pytest                              # all tests
pytest tests/test_semantic_routing.py -v   # one ticket at a time
pytest -k "force_flag" -v           # by name pattern
```

The tests **do not** require a real Elasticsearch — `tests/conftest.py` provides `FakeAsyncES` (in-memory async stub) and `FakeEmbedder` (deterministic SHA-derived vectors). Integration tests against a real cluster should live in a separate `tests/integration/` folder run from CI with docker-compose.

> **The author of this drop did not run pytest** — the iteration environment had no internet and no preinstalled deps, so the test files are correct-by-inspection only. The first thing you should do is `pip install -r requirements.txt && pytest`. If anything breaks it will almost certainly be in `tests/conftest.py::FakeAsyncES` (the surface most likely to drift from the real `elasticsearch[async]` client).

Known gaps / follow-ups

- **No retries / circuit breaking** on the embedder. Tenacity around `RouterEmbedder.embed` is the first thing I'd add for production.
- **`tiktoken`-accurate token counts** are not in this drop. The POC's `len(text)//3` heuristic was a footgun at scale; replace before turning on the real router under load.
- **RRF (reciprocal rank fusion)** is the right hybrid scoring primitive on ES 8.9+. Current code uses simple boost weighting because it works on older clusters. Swap when you confirm cluster version.
- **Audit log table (DADV-3387)** is not implemented — needs a Postgres schema decision first. The structured JSON access log is the placeholder.
- **`AsyncElasticsearch` lifecycle** — currently a process-global lazy singleton. Fine for uvicorn workers, would need adjustment for testing alternative ASGI servers.

Layout

```
rag_service/
├── app/
│   ├── config.py              # METADATA_SCHEMA + Settings  ← single source of truth
│   ├── hashing.py             # SHA-256 over normalized text
│   ├── embeddings.py          # Embedder protocol + httpx implementation
│   ├── es_client.py           # AsyncElasticsearch factory
│   ├── exceptions.py          # RagError hierarchy
│   ├── index_manager.py       # ensure_index() + build_index_mapping()
│   ├── ingestion.py           # ingest_chunks() with dedup
│   ├── search.py              # hybrid_search() + build_hybrid_query()
│   ├── models.py              # Pydantic request/response models
│   ├── logging_config.py      # JSON logs + correlation IDs
│   ├── middleware.py          # FastAPI middleware + error handlers
│   └── routers/
│       ├── ingestion_router.py
│       └── search_router.py
├── tests/
│   ├── conftest.py            # FakeAsyncES, FakeEmbedder, app_client
│   ├── test_semantic_routing.py
│   ├── test_dynamic_metadata.py
│   ├── test_ingestion_api.py
│   ├── test_ingestion_errors.py
│   ├── test_search_api.py
│   └── test_search_errors.py
├── main.py                    # FastAPI app + lifespan
├── requirements.txt
├── pytest.ini
├── .env.example
└── README.md
```